به نام خدا

ساختمان داده‌ها و الگوریتم‌ها (۴۰۲۵۴)

دانشگاه صنعتی شریف — تمرین دوم

مدرس: مهدی صفرنژاد — انتشار: ۶ اسفند ۹۷

مرتب‌سازی

سؤال ۱. **[۸ نمره]** دو تابع F و G زیر را به کمک درخت بازگشت به صورت مجانبی تحلیل کنید.

$$F(۰) = G(۰) = ۰$$
$$F(n) = F(\lfloor \frac{۲n}{۳} \rfloor) + G(n)$$
$$G(n) = F(\lfloor \frac{n}{۳} \rfloor) + n$$

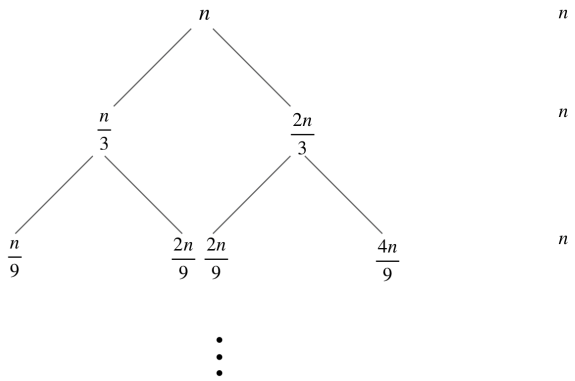
پاسخ: با جایگذاری رابطه دوم در رابطه اول داریم:

$$F(n) = F(\lfloor \frac{۲n}{۳} \rfloor) + F(\lfloor \frac{n}{۳} \rfloor) + n$$

برای تحلیل مجانبی تابع بالا می‌توانیم، رابطه بالا را به شکل زیر بازنویسی کنیم:

$$F(n) = F(\frac{۲n}{۳}) + F(\frac{n}{۳}) + n$$

درخت بازگشت این رابطه را می‌توان به شکل زیر ترسیم کرد. دقت کنیم که عمق برگ‌های این درخت حداکثر $\log_{\frac{۳}{۲}} n$ و حداقل



شکل ۱: درخت بازگشت F

$\log_۳ n$ است. پس به سادگی می‌توان دید:

$$\underbrace{n+n+\cdots+n}_{\log_۳ n} \le F(n) \le \underbrace{n+n+\cdots+n}_{\log_{\frac{۳}{۲}} n}$$

به عبارت دیگر

$$n\log_۳ n \le F(n) \le n\log_{\frac{۳}{۲}} n$$

پس بنا بر رابطه تغییر پایه در لگاریتم‌ها داریم:

$$c_۱ n\log_۲ n \le F(n) \le c_۲ n\log_۲ n$$

که نشان می‌دهد $F(n) \in \Theta(n\log n)$. همچنین با توجه به رابطه‌ای که برای G داریم

$$c_۱\frac{n}{۳}\log_۲\frac{n}{۳} + n \le G(n) = F(\frac{n}{۳}) + n \le c_۱\frac{n}{۳}\log_۲\frac{n}{۳} + n$$

که نشان می‌دهد:

$$\frac{c_۱}{۳}n\log_۲ n - c'_۱ n \le G(n) \le \frac{c_۲}{۳}n\log_۲ n - c'_۲ n$$

پس

$$G(n) \in \Theta(n\log n)$$

سؤال ۲. [۱۰+۱۰ نمره]

آ. داده ساختاری طراحی کنید که در زمان $O(۱)$ عنصر i ام لیست را بیابد و اعمال درج و حذف عناصر ابتدا و انتها را زمان سرشکن $O(۱)$ انجام دهد.

ب. با ادغام داده ساختارهای صف و پشته، داده ساختاری ساختیم که عملیات زیر را پشتیبانی می‌کند.

- Pop: حذف کردن یک عنصر از ابتدای سمت راست لیست.
- Push: اضافه کردن یک عنصر به ابتدای سمت راست لیست.
- Pull: حذف کردن یک عنصر از ابتدای سمت چپ لیست.

با استفاده از تنها ۳ پشته و $O(۱)$ حافظه اضافی داده ساختار را طوری طراحی کنید که اعمال گفته‌شده در زمان سرشکن $O(۱)$ قابل انجام باشد.

پاسخ:

- برای این سوال راه حل ها متفاوتی وجود دارد. برای ذخیره سازی n عنصر یک آرایه به طول $۳n$ را در نظر می گیریم و عناصر را در وسط آن قرار می‌دهیم. به این ترتیب عملیات پیدا کردن عنصر در زمان $O(۱)$ انجام پذیر است همچنین برای درج

عنصر در ابتدا کافیست عنصر را به سمت چپ داده‌ها اضافه کنیم. به همین ترتیب می توانیم عناصر را در انتها داده‌ها اضافه کنیم. اگر هنگام درج فضای اضافه‌ای وجود نداشت یک آرایه جدید به طول سه برابر تعداد عناصر کنونی درست می کنیم و آنها را در وسط آرایه جدید قرار می دهیم و آرایه قبلی را حذف می کنیم. توجه کنید تعداد درج‌هایی که در آرایه قبلی انجام داده‌ایم قبل از این‌که آن را بازسازی کنیم خطی بوده است. همچنین برای حذف هنگامی که تعداد عناصر از یک ششم طول آرایه کمتر شد دوباره همین کار را تکرار می‌کنیم. توجه کنید تعداد حذف‌هایی که در آرایه قبلی پیش از بازسازی آن انجام شده است خطی می‌باشد. به این ترتیب اعمال درج و حذف عناصر ابتدا و انتها را زمان سرشکن $O(1)$ قابل انجام است.

ذکر این نکته لازم است که برای دسترسی $O(1)$ به عناصر کافی است اندیس اولین عنصر و تعداد عناصر را نگه‌دار و به روز کنیم.

- نصف ابتدای عناصر را در یک پشته و نصف انتها را به صورت برعکس در یک پشته دیگر نگه داری می کنیم به صورتی که عناصر اول و آخر در بالای پشته‌ها قرار گیرند. در این صورت عملیات Push در $O(1)$ قابل انجام است. هنگام Pop و Pull اگر هر دو پشته پر باشند می‌توانیم این عملیات را در $O(1)$ انجام دهیم. اگر یکی از پشته‌ها خالی شد (به طور مثال پشته دوم) نصف عنصرهای پشته پر (به طور مثال پشته اول) را Pop کرده و در پشته سوم می ریزیم. سپس از پشته اول دوباره نصف باقی مانده را Pop می‌کنیم و در پشته دوم می ریزیم. سپس تمام عناصر موجود در پشته سوم را Pop می‌کنیم و در پشته اول می ریزیم به این ترتیب عنصر ابتدا و انتها دوباره در بالای پشته دوم و اول قرار می گیرند. همچنین تعداد عملیات انجام شده قبل از هر بازسازی خطی است در نتیجه می توان تمام این عملیات را در زمان سرشکن $O(1)$ انجام داد.

سؤال ۳. [۱۰+۱۰ نمره]

آ. الگوریتم مرتب سازی زیر را در نظر بگیرید (این الگوریتم مرتب سازی به selection sort معروف است)

```
def min_index(a, i, j):
        if i == j:
                return i
        k = min_index(a, i + 1, j)

        if a[i] < a[k]:
                return i
        else:
                return k


def selection_sort(a, n, index):
        if index == n:
                return -1
        k = min_index(a, index, n - 1)

        if k != index:
                a[k], a[index] = a[index], a[k]
        selection_sort(a, n, index + 1)
```

```
20
21
22 def main():
23         arr = [32, 11, -5, 1, 7, 61, 43, 3]
24         selection_sort(arr, len(arr), 0)
25
26 main()
```

۱. درستی این الگوریتم را ثابت کنید.

۲. زمان اجرای آن را تحلیل کنید.

ب. یک مدل از مرتب‌سازی merge sort برای یک آرایه A به صورت زیر می‌باشد:(منظور از $A[i..j]$ بخشی از آرایه که بین $A[i]$ و $A[j]$ هست می‌باشد)

- اگر $n = ۱$ در آن صورت الگوریتم به پایان می‌رسد.
- در غیر اینصورت قرار دهید $k = \lceil \frac{n}{۳} \rceil$
- به صورت بازگشتی، $A[۱..k]$ و $A[k+۱..n]$ را مرتب کنید.
- دو زیرآرایه $A[۱..k]$ و $A[k+۱..n]$ را ادغام کنید.

ثابت کنید پیچیدگی زمانی این الگوریتم برابر است با $\mathcal{O}(n \log n)$.

پاسخ:

- در واقع این الگوریتم در مرحله kاُم عدد kاُم را (از لحاظ کوچکی) سر جای خودش قرار می دهد پس بعد از n مرحله هر عددی سر جای خودش قرار می‌گیرد. (در مرحله اول عنصر کمینه را در جایگاه اول قرار می دهد سپس در گام بعدی عنصر کمینه بعدی را در جایگاه دوم قرار می دهد و ... تا به آخر آرایه برسد)

  به ازای هر حالت اولیه‌ای در زمان $O(n^۲)$ آرایه را مرتب می‌کند. چون هر دفعه باید از بین k عدد، کمینه را پیدا کند $(۱ \leq k \leq n)$ پس تعداد عملیات انجام شده برابر است با $۱ + ۲ + ... + n = O(n^۲)$.

- هر دفعه آرایه به دو قسمت $\lceil \frac{n}{۳} \rceil$ تایی و $\lfloor \frac{۲n}{۳} \rfloor$ تایی تقسیم می‌شود. پس رابطه بازگشتی زمان اجرای این الگوریتم برابر است با $T(n) = T(\lceil \frac{n}{۳} \rceil) + T(\lfloor \frac{۲n}{۳} \rfloor) + O(n)$. (توجه کنید ادغام کردن مانند ادغام merge sort معمولی در $O(n)$ انجام می شود)

  با ترسیم درخت بازگشت برای این رابطه بازگشتی متوجه می شویم که جواب آن $O(n \log n)$ است چون تعداد طبقات $\log_{۱/۳} n$ است و در هر طبقه عملیات در مجموع از مرتبه $O(n)$ است.

سؤال ۴. [۱۲ نمره] توابع زیر را به صورت مجانبی تحلیل کنید و به صورت کامل توضیح دهید. برای آشنایی بیشتر و مرور روش‌های تحلیل می‌توانید به بخش ۳ـ۵ کتاب داده‌ساختارها و مبانی الگوریتم‌ها مراجعه کنید.

آ. $T(n) = T(\lfloor \frac{n}{۲} \rfloor) + T(\lceil \frac{n}{۲} \rceil) + n$

پاسخ: از حدس و استقرا برای حل این مسئله استفاده می‌کنیم. $T \in \Theta(n\log(n))$ می‌تواند با توجه به رابطهٔ بازگشتی، حدسی منطقی باشد. فرض می‌کنیم برای b و c و n های کوچکتر از m داریم:

$$T(n) \leq bn\log(n) + c$$

می‌خواهیم داشته باشیم:

$$T(m) = T(\lfloor \frac{m}{۲} \rfloor)+T(\lceil \frac{m}{۲} \rceil)+m \leq b(\lfloor \frac{m}{۲} \rfloor \log(\lfloor \frac{m}{۲} \rfloor)+\lceil \frac{m}{۲} \rceil \log(\lceil \frac{m}{۲} \rceil))+۲c+m \leq bm\log(m)+c$$

$$\Leftrightarrow b(\lfloor \frac{m}{۲} \rfloor+\lceil \frac{m}{۲} \rceil)\log(\frac{m+۱}{۲})+c+m \leq bm\log(m) = bm\log(\frac{m+۱}{۲})+c+m \leq bm\log(m)$$

$$\Leftrightarrow ۱ + \frac{c}{m} \leq b\log(\frac{۲m}{m+۱})$$

برای این که نابرابری بالا برقرار باشد کافی است c را منفی در نظر بگیریم و b را به اندازه‌ی کافی بزرگ. در این صورت طبق استقرا برای هر n خواهیم داشت:

$$T(n) \leq bn\log(n) + c \Rightarrow T \in O(n\log(n))$$

به همین صورت می‌توان ثابت کرد که $T \in \Omega(n\log(n))$. در نتیجه $T \in \Theta(n\log(n))$.

ب. $T(n) = T(n-۱) + \frac{۱}{n}$

پاسخ:

داریم:

$$\frac{۱}{۲} \leq \sum_{j=۲^i}^{۲^{i+۱}-۱} \frac{۱}{j} \leq ۱$$

$$\Rightarrow \frac{\lfloor \log_۲(n) \rfloor}{۲} \leq T(n) = \sum_{i=۱}^{n} \frac{۱}{i} \leq \lceil \log_۲(n) \rceil$$

$$\Rightarrow T \in \Theta(\log(n))$$

پ. $T(n) = ۲T(\sqrt[۳]{n}) + \log(n)^۲$

پاسخ: از تغییر متغیر $m = log(n)$ استفاده می‌کنیم و سپس قرار می‌دهیم $S(m) = T(۲^m)$. در این صورت خواهیم داشت:

$$S(m) = ۲S(\frac{m}{۳}) + m^۲$$

طبق قضیه‌ی اصلی $S \in \Theta(m^۲)$. پس $T \in \Theta(log(m)^۲)$. با جایگذاری داریم $T \in \Theta(log(n)^۲)$.

ت. **قضیه‌ی اصلی:** در این قسمت مشخص کنید که هر کدام از توابع بازگشتی با قضیه‌ی اصلی قابل تحلیل است یا خیر. در صورتی که تابعی با قضیه‌ی اصلی قابل تحلیل نباشد نیاز به تحلیل آن در این قسمت نیست.

- $T(n) = ۳T(\frac{n}{۵}) + n^{۰/۶}$
- $T(n) = ۳T(\frac{n}{۳}) + \frac{n}{log(n)}$

پاسخ:

- $n^{\log_b a} = n^{۰/۶۸۲}$ و چون $n^{۰/۶}$ از $n^{۰/۶۸۲}$ به صورت چند جمله ای کوچکتر است پس در این حالت جواب $\theta(n^{۰/۶۸۲})$ است. (حالت اول قضیه اصلی)
- $n^{\log_b a} = n^{۱}$ و چون $\frac{n}{log(n)}$ از n به صورت چند جمله ای کوچکتر نیست پس در این حالت نمی توان از قضیه اصلی استفاده کرد.

سؤال ۵. [۲۵ نمره] گادوین منشا، به تازگی از تیم سابقش کناره‌گیری کرده و برای کار کردن در یک موسسه آموزشی استخدام شده است. کاری که او همواره می‌کند به این شرح است: در روز به او تعدادی درخت دودویی داده می‌شود و او باید با جابه‌جایی تعدادی از راس‌ها کاری کند که این درخت دودویی «بارِ دانش» بگیرد. درخت‌هایی که «بارِ دانش» گرفته‌اند، در حقیقت درخت دودویی جستجو هستند. از آن‌جا که منشا اخیرا به دانشگاه شریف آمده و دوستانی پیدا کرده‌است، از آن‌ها خواسته برایش برنامه‌ای بنویسند که این کار را برایش انجام دهد تا از کار بیکار نشود.

ورودی:

در خط اول ورودی ابتدا n می‌آید که نشان‌دهنده تعداد راس‌های درخت دودویی اولیه است.
در خط دوم ورودی n عدد می‌آیند که نشان‌دهنده مقدار هر گره درخت دودویی است.
نحوه ورودی دادن درخت به این‌ترتیب است که اگر ورودی i اُم و $2i$ اُم و $2i+1$ اُم به ترتیب فرزند چپ و راست ورودی iاُم هستند.

خروجی:

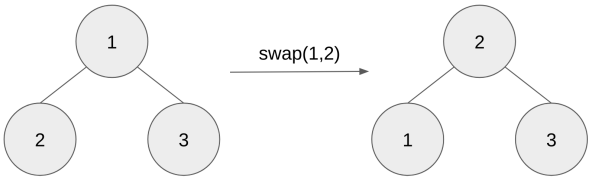
در تنها خط خروجی، کم‌ترین تعداد جابه‌جایی‌ها برای تبدیل درخت دودویی به ددج را چاپ کنید.

محدودیت‌ها:

تمام ورودی‌ها از 10^5 کم‌تر هستند.

ورودی و خروجی نمونه:

stdin	stdout
3 1 2 3	1



پاسخ: در ابتدا گراف را از روی ورودی مسئله می‌سازیم، می‌دانیم که اگر قرار باشد این گراف، یک درخت دودویی جستجو باشد، پیمایش میان‌ترتیب [1] آن باید یک آرایه مرتب شده باشد. بنابراین مسئله تبدیل می‌شود به این که پیمایش میان‌ترتیب درخت مورد نظر را با کم‌ترین تعداد جابجایی، به یک آرایه مرتب شده تبدیل کنیم. بدین منظور، فرض کنید که در آرایه مورد نظر از هر عددی در آرایه، به سمت مکانی که این عدد بعد از مرتب شدن قرار می‌گیرد یک یال جهتدار داریم. در این صورت یک گراف جهتدار که از تعدادی مولفه همبندی تشکیل شده‌است داریم، در این صورت کم‌ترین تعداد جابجایی‌ها در هر مولفه همبندی برای مرتب شدن برابر می‌شود با تعداد راس‌های مولفه همبندی منهای ۱، حال با جمع این مقادیر، مقدار مربوطه بدست می‌آید. همچنین کد موردنظر این سوال، پاسخ ارسالی آقای علیرضا حاجلی می‌باشد که در فایل `mensha.py` موجود است.

[1] inorder

سؤال ۶. [۲۵ نمره] altF4، قهرمان داستان ما، علاقه بسیار زیادی به بازی کردن با آرایه‌ها دارد و تخصص اصلی او در مرتب کردن آرایه هاست. به تازگی دوستان او، altF4 را با یک چالش جدید روبرو کرده‌اند. چالش به این صورت است که در ابتدا به او یک آرایه به طول Nاز اعداد صحیح داده می‌شود و وظیفه altF4 این است که این آرایه را به صورت صعودی مرتب کند. اما مشکل اصلی این است که دوستانش یک سری شرط برای این کار گذاشته‌اند. از جمله این که altF4 در هر حرکت می‌تواند جای دو عنصر مجاور a[i] و a[i+1] را با هم عوض کند.(به ازای $1 \leq i \leq N-1$ ،اولین عنصر آرایه a[1] در نظر گرفته شده است) دوستان altF4 پا را فراتر گذاشته‌اند و این شرط را هم اضافه کرده‌اند که با جابه‌جا کردن a[i] و a[i+1]، مقدار a[i] یک واحد کاهش و مقدار a[i+1] یک واحد افزایش می‌یابد. او عملیات جابه‌جایی را به هر تعداد که بخواهد، می‌تواند انجام دهد. از آنجا که altF4 به همه گفته که تخصصش، مرتب کردن آرایه‌هاست و الان از پس این کار برنمی‌آید، به سراغ شما آمده تا مشکل او را برطرف کنید.

ورودی:

در خط اول ورودی اندازه آرایه ورودی یعنی N می‌آید.

در خط دوم ورودی N عدد که اعضای آرایه هستند با یک فاصله از هم می‌آیند.

خروجی:

در خروجی، اگر آرایه مرتب‌شده وجود نداشت، عبارت No چاپ می‌شود.

اگر آرایه مرتب‌شده وجود داشت، در خط اول عبارت Yes و در خط بعدی آرایه مرتب شده چاپ می‌شود.

محدودیت‌ها:

$1 \leq N \leq 10^5$

$1 \leq a[i] \leq 10^9$

ورودی و خروجی نمونه:

input	output
4 7 3 9 10	Yes 4 6 9 10

input	output
3 11 1 9	No

پاسخ: برای حل سوال همه اعضای آرایه را به خانه صفر آرایه منتقل میکنیم. اعداد آرایه را در خانه صفر مرتب می‌کنیم. مجدداً اعداد را به ترتیب در آرایه درج می‌کنیم.

در آخر در صورت مرتب بودن آرایه نهایی این عملیات انجام پذیر است.همچنین کد پیاده‌سازی نیز در فایل altF4.py موجود است.

موفق باشید